REPORT 10 - DevOps Plan

Levi Nelson, Carlos W. Mercado

Brigham Young University-Idaho

CSE 272 - Software Lifecycle Models

Instructor: Br. Edward Bullen

Date: 03/08/2025

1. OVERVIEW

1.1 Project Overview

Bank Associate Nation Keeper, Inc (B.A.N.K., Inc) has hired our company for building a mobile application that will help homeowners stay on top of their home maintenance. The mobile application includes features as follows:

- Monitors electricity usage,
- Monitor appliance upkeep,
- Monitor yard maintenance,
- Track periodic cleaning.

The application will provide a monthly checklist of home maintenance activities, allow the homeowner to check off items when they are complete, and produce a report at the end of the month.

Due to the Agile methodology's emphasis on working software (Beck et al. 2001), and the Scrum Management Framework practice of "incremental, iterative work cadences, known as sprints" (J. Girish, 2013), our software will be developed through several sprints, each concluding with deployment (James, 2012). Our project will consist of five 3-week iterations, resulting in a total time of approximately 15 weeks.

1.2 Project Specifics and Assumptions

- We are building a mobile application.

- Business requirements have been included in the contract.
- Pre-contract meetings took place with customer representatives.
- More meetings will be held throughout the development of the application, including a *Product Requirements Meeting,* following best Scrum practices and guidelines.

1.3 Implemented Software Development Method: Using DevOps

1.3.1 Agile Software Development Methodology & Scrum Management Framework

We will be building the current plan of development over our previous report “REPORT 08 - Scrum Plan” (Nelson & Mercado, 2025). Any reference to Agile or Scrum techniques made in the present report, should be consulted in that report. Here, we will discuss their implementation following the DevOps practices.

1.3.1.1 How do DevOps and Agile relate to one another?

We subtract the following description from Microsoft Azure DevOps Tutorial (2025) :

- Although both DevOps and agile are software development practices, they each have a slightly different focus.
- DevOps is a culture that focuses on creating efficiency for all stakeholders involved in the *development*, *deployment*, and *maintenance* of software.
- Agile, on the other hand, is a lean manufacturing process that helps provide a software development production framework.
- Agile is often specific to the development team, where the scope of DevOps extends to all stakeholders involved in the production and maintenance of software.

- DevOps and agile can be used together to create a highly efficient software development environment.

1.3.2 DevOps Development Method: Overview

DevOps combines development (Dev) and operations (Ops) to unite people, process, and technology in application planning, development, delivery, and operations. DevOps enables coordination and collaboration between formerly siloed roles like development, IT operations, quality engineering, and security (Jacobs et al., 2023).

Teams adopt DevOps culture, practices, and tools to increase confidence in the applications they build, respond better to customer needs, and achieve business goals faster. DevOps helps teams continually provide value to customers by producing better, more reliable products (Jacobs et al., 2023).

For the development of our plan we will focus on more established trends that focus on more widely known concepts and aspects of DevOps (see Figure 1), and we will only succinctly mention other more recent transitional variations, namely DevSecOps. For the latter purpose we've included Figure 2 and Figure 3 in the appendix section of this report (Kukushkina, 2023) for further reference.

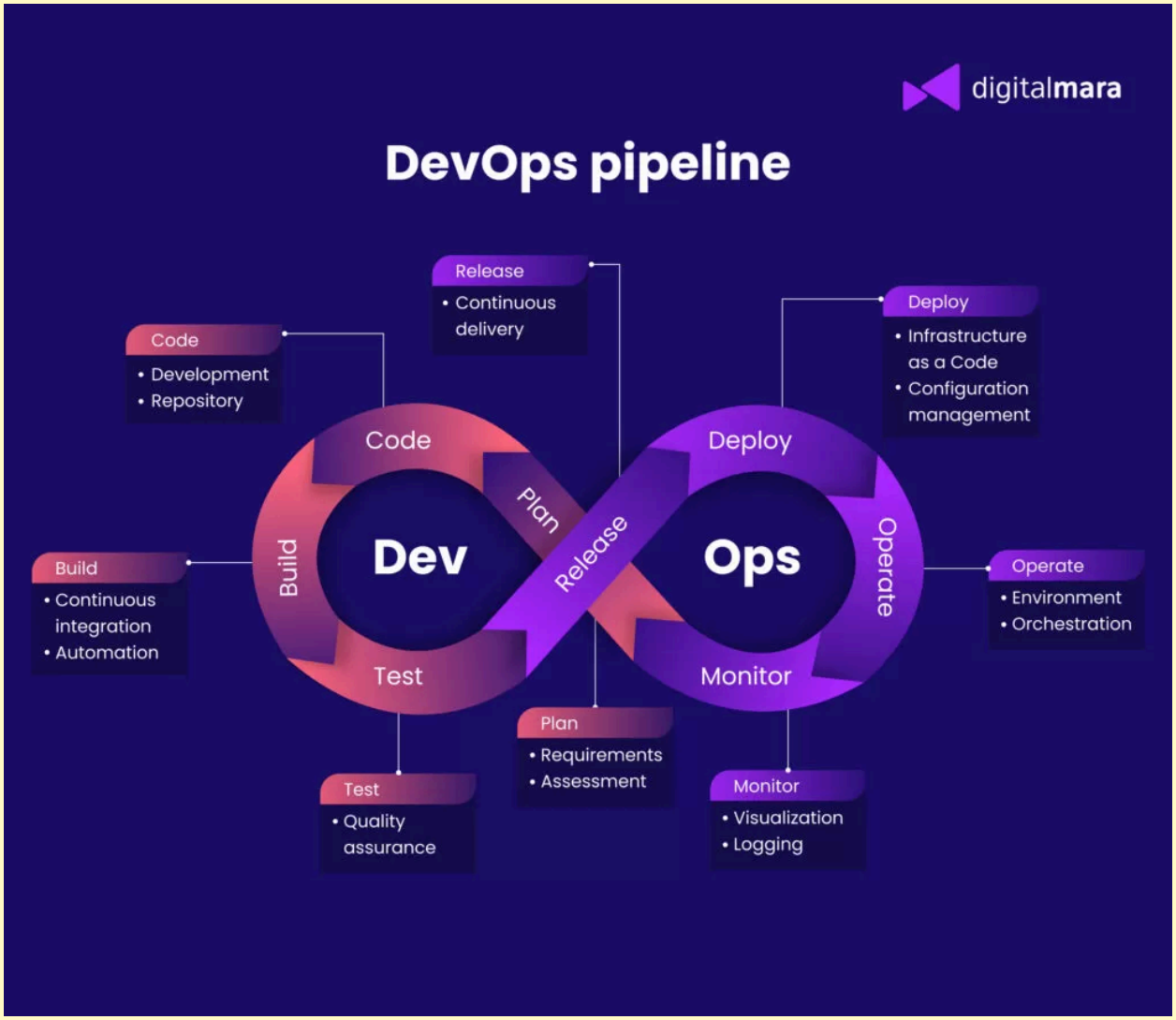

Figure 1 *- DevOps Pipeline description (Kukushkina, 2023). In recent years, the development of this development method has seen the rise of DevSecOps, which integrates "Security" features into "Development & Operations" (see Figures 2 and 3 in appendix section).*

The following practices are key components of a DevOps culture (Jacobs et al., 2023):

- **Collaboration, visibility, and alignment:** A hallmark of a healthy DevOps culture is collaboration between teams. Collaboration starts with visibility. Development, IT, and other teams should share their DevOps processes, priorities, and concerns with each other. By planning their work together, they are better positioned to align on goals and measures of success as they relate to the business.

- **Shifts in scope and accountability:** As teams align, they take ownership and become involved in other lifecycle phases—not just the ones central to their roles. For example, developers become accountable not only to the innovation and quality established in the develop phase, but also to the performance and stability their changes bring in the operate phase. At the same time, IT operators are sure to include governance, security, and compliance in the plan and develop phase.
- **Shorter release cycles:** DevOps teams remain agile by releasing software in short cycles. Shorter release cycles make planning and risk management easier since progress is incremental, which also reduces the impact on system stability. Shortening the release cycle also allows organizations to adapt and react to evolving customer needs and competitive pressure.
- **Continuous learning:** High-performing DevOps teams establish a growth mindset. They fail fast and incorporate learning into their processes. They strive to continually improve, increase customer satisfaction, and accelerate innovation and market adaptability.

1.3.3 DevOps Practices

In the following explanation we will primarily follow the explanation provided by Jacobs (2023) while using to some extent the classification provided by Kukushkina (2023) in Figure 1 (translated into Table 1).

Lifecycle Stage	**Pipeline Stage Name**	**DevOps Practice**	**Jacobs Correlation (possible correlations)**
1	PLAN	• Requirements • Assessment	• Agile Software Development
2	CODE	• Development • Repository	• “Repository”, correlates with “Version Control”
3	BUILD	• Continuous Integration (CI) • Automation	-
4	TEST	• Quality Assurance	-
5	RELEASE	• Continuous Delivery (CD)	-
6	DEPLOY	• Infrastructure as Code (IaC)	-
7	OPERATE	• Environment • Orchestration	• Configuration Management
8	MONITOR	• Visualization • Logging	• Continuous Monitoring

Table 1 *- DevOps Lifecycle Stages and Practices (based on Figure 1).*

Every practice in Table 1 is placed inside the DevOps pipeline (as displayed in Figure 1), while every stage is a component to the overall software development lifecycle of DevOps (Kukushkina, 2023). In the table, between square brackets we included comments that help the alignment of the table with the description provided by Jacobs et al. (2023).

1.3.3.1 Continuous integration and continuous delivery (CI/CD)

- **CONTINUOUS INTEGRATION or CI (BUILD STAGE)**: The practice used by development teams to automate, merge, and test code. CI helps to catch bugs early in the development cycle, which makes them less expensive to fix. Automated tests execute as part of the CI process to ensure quality. CI systems produce artifacts and feed them to release processes to drive frequent deployments.
- **CONTINUOUS DELIVERY or CD (RELEASE STAGE)**: The process by which code is built, tested, and deployed to one or more test and production environments. Deploying and testing in multiple environments increases quality. CD systems produce deployable artifacts, including infrastructure and apps. Automated release processes consume these artifacts to release new versions and fixes to existing systems. Systems that monitor and send alerts run continually to drive visibility into the entire CD process.

1.3.3.2 Version Control (CODE STAGE)

- Version control is the practice of managing code in versions—tracking revisions and change history to make code easy to review and recover. This practice is usually implemented using version control systems such as Git, which allow multiple developers to collaborate in authoring code. These systems provide a clear process to merge code changes that happen in the same files, handle conflicts, and roll back changes to earlier states.

1.3.3.3 Agile Software Development (PLAN STAGE)

- Agile is a software development approach that emphasizes team collaboration, customer and user feedback, and high adaptability to change through short release cycles. Teams

that practice Agile provide continual changes and improvements to customers, collect their feedback, then learn and adjust based on customer wants and needs. Agile is substantially different from other more traditional frameworks such as waterfall, which includes long release cycles defined by sequential phases. Kanban and Scrum are two popular frameworks associated with Agile.

1.3.3.4 Infrastructure as Code or IaC (DEPLOY STAGE)

- Infrastructure as code defines system resources and topologies in a descriptive manner that allows teams to manage those resources as they would code.
- Practicing infrastructure as code helps teams deploy system resources in a reliable, repeatable, and controlled way. Infrastructure as code also helps automate deployment and reduces the risk of human error, especially for complex large environments. This repeatable, reliable solution for environment deployment lets teams maintain development and testing environments that are identical to production. Duplicating environments to different data centers and cloud platforms likewise becomes simpler and more efficient.

1.3.3.5 Configuration Management (OPERATE STAGE)

- Configuration management refers to managing the state of resources in a system including servers, virtual machines, and databases. Using configuration management tools, teams can roll out changes in a controlled, systematic way, reducing the risks of modifying system configuration. Teams use configuration management tools to track system state and help avoid configuration drift, which is how a system resource’s configuration deviates over time from the desired state defined for it.

- Along with infrastructure as code, it's easy to templatize and automate system definition and configuration, which help teams operate complex environments at scale.

1.3.3.6 Continuous Monitoring (MONITOR STAGE)

- Continuous monitoring means having full, real-time visibility into the performance and health of the entire application stack. This visibility ranges from the underlying infrastructure running the application to higher-level software components. Visibility is accomplished through the collection of telemetry and metadata and setting of alerts for predefined conditions that warrant attention from an operator. Telemetry comprises event data and logs collected from various parts of the system, which are stored where they can be analyzed and queried.
- High-performing DevOps teams ensure they set actionable, meaningful alerts and collect rich telemetry so they can draw insights from vast amounts of data. These insights help the team mitigate issues in real time and see how to improve the application in future development cycles.

1.3.3.6 OTHER PRACTICES

As per the current state of art, and the novelty and evolving nature of DevOps in the last couple of years (as explained by Jabbari et al., 2016, “Given that DevOps is a new term and novel concept recently introduced, no common understanding of what it entails has been achieved yet. Consequently, definitions of DevOps often only represent a part that is relevant to the concept”), it is necessary to briefly mention and cover additional practices implemented in DevOps. These practices may also be incorporated during the adoption of this development methodology in our project.

Some of this other practices are:

- **AUTOMATION** (Kolodiy, 2023): It involves streamlining labor-intensive tasks like testing, deployment, and software updates through automated processes. The impact is profound – it doubles productivity, effortlessly detects and remedies bugs, and injects a new level of efficiency into development.

  Furthermore, automation is a potent weapon against human errors in the software delivery process. It can seamlessly handle essential tasks, such as installing security updates across your entire environment, from workstations to servers and software components, ensuring everything remains up-to-date without constant human intervention.

- **COLLABORATION** (Kolodiy, 2023): DevOps thrives on collaboration, creating a dynamic partnership between IT and software development departments. This collaboration is built on sharing ideas and providing continuous feedback.
- **MONITORING AND LOGGING** (Sudo Consultants, 2024): Both logging and monitoring are necessary in DevOps to keep up a continuous delivery pipeline. Monitoring is the act of gathering and examining data from applications and systems to make sure they function as intended. On the other hand, logging entails keeping track of things that happen within a program or system. Together, these two techniques enable teams to address problems promptly and minimize downtime.
- **MICROSERVICES** (Kolodiy, 2023): Instead of building monolithic applications, DevOps encourages breaking them down into smaller, self-contained services. This modular approach enables flexibility, scalability, and agility in application development, allowing teams to innovate and deploy more efficiently.

- **FEEDBACK LOOPS** (Krammer, 2024): Integrating feedback loops into your CI/CD pipeline is crucial for improving product quality, development speed, and team productivity. By implementing effective feedback mechanisms, developers can identify and address issues early, reducing the likelihood of downstream problems and improving overall development efficiency. To implement and maintain effective feedback loops within CI/CD practices, remember the following practices:
    - Automated Testing - Catch bugs and defects early, reducing manual testing and debugging.
    - Continuous Monitoring - Get real-time insights into pipeline performance, identifying bottlenecks and optimizing processes.
    - Security and Compliance - Integrate security and compliance checks into feedback loops to ensure code integrity and security.
    - Peer Review and Collaboration - Use collaboration tools to facilitate seamless communication and feedback among team members, promoting continuous improvement.
    - Choosing the Right CI/CD Tools - Select tools that facilitate efficient feedback mechanisms, align with team and project requirements, and improve the overall development process.
- **DEPLOYMENT PIPELINES** (Krammer, 2024a): A CI/CD pipeline is a set of automated steps that help move code from development to production. It combines continuous integration (CI) and continuous delivery or deployment (CD) to speed up software development. The pipeline helps teams to: Improve software quality, Release updates faster, Reduce errors and downtime.

- **SCALABILITY** (Wasike, 2023): DevOps facilitates scalability by leveraging technologies like containerization and infrastructure as code. Organizations can easily scale their infrastructure and adapt to changing requirements with minimal effort.
- **RESILIENCE** (Codecademy, 2025): In DevOps, resiliency is a system's ability to continue to provide an acceptable level of performance even when problems are occurring within the system. Common problems that occur to a system are internal failures, external failures, as well as malicious attacks. These problems are all inevitable, but DevOps practices can minimize their frequency and severity. Resiliency can be achieved through methods such as: Caching, Input validation, Load balancing.

1.3.4 DevOps Summary

The following items summarize the previous descriptions:

- DevOps is described as by different authors as:
    - Software Development Methodology. "The DevOps methodology aims to shorten the systems development lifecycle and provide continuous delivery with high software quality" (GitLab, 2023).
    - "Collaborative organizational model that brings together software development and operations teams" (Queen, 2024).
    - "DevOps, the combination of Development and Operations, is a new way of thinking in the software engineering domain that recently received much attention. Given that DevOps is a new term and novel concept recently introduced, no common understanding of what it entails has been achieved yet.

> Consequently, definitions of DevOps often only represent a part that is relevant to the concept" (Jabbari et al., 2016)

- As explained in the previous paragraph, DevOps is a relatively "new way of thinking" development, and for that reason it is quickly evolving into new versions, including changes with new practices and points of view. For these reasons, there is not a wide and established consensus over what DevOps is and how it should be implemented, which leads to a scenario where different authors (individuals, people, organizations) present DevOps in different ways.

- DevOps entices multiple practices, and though some authors provide with success complex taxonomies of these (see next sub-items), we've decided to reduce them to the more widely known and used in the industry. We've described what we consider the basic DevOps practices in section **1.3.3** of this report.

    - As Jabbari et al. (2016) explains: "After identifying these [DevOps] practices, we have categorized them according to the fundamental knowledge areas, and corresponding sub-knowledge areas, proposed in software engineering body of knowledge (Swebok), as an international standard for providing a comprehensive categorized collection of the bounds of software engineering."

    - Following this explanation, we've decided to reduce the scope of our report to the more common practices.

- Because of DevOps evolving nature, in recent years we've seen the rise of DevSecOps, which is an iteration of DevOps that integrates security practices into the DevOps workflow. "Because both models share cultural similarities and focus on collaboration

and automation, it can be easy to confuse them, but they address different business goals. A helpful way of thinking of DevOps vs. DevSecOps is that all DevSecOps teams use DevOps, but not all DevOps teams use DevSecOps" (Queen, 2024).

Keywords: software development methodologies, Agile, DevOps, DevSecOps, Operations, Development, management framework Scrum

2. MEETINGS

2.1 Product Requirements Meeting

2.1.1 ***Who***

- *ScrumMaster, Product Owner, Stakeholders*

2.1.2 ***Agenda***

- *Welcome*
- *Discussion of project specifics*
- *Definition of software requirements*
- *Documentation of user stories*
- *Creation of Product Requirements Document*

2.1.3 ***Purpose***

- *The purpose of the Product Requirements Meeting is to identify the specifics of the project to be reached over several iterations. Working with the stakeholders, project and software requirements will be identified and documented. User stories will also be discussed and documented.* This meeting will result in the creation of the Product Requirements document, which will contain this information for reference over the course of the project (Radigan n.d). *Though the Products Requirement Meeting is not required in James's Scrum Reference Card, we have*

chosen to implement a Products Requirement Document and Meeting for the mutual benefit of our stakeholders and company.

2.1.4 ***Event***

- *Held once at the beginning of the project.*

2.2 Sprint Planning Meeting

2.2.1 ***Who***

- *All employees*

2.2.2 ***Agenda***

- *Welcome*
- *Backlog refinement (as needed; only necessary if the Backlog Refinement Meeting was insufficient)*
- *Declaration of important Backlog Items*
- *Backlog Item selection*
- *Conversion of Backlog Items into Sprint Tasks.*

2.2.3 ***Purpose***

- The purpose of the Sprint Planning Meeting is to select which Product Backlog Items will be converted into products during the sprint. The Product Owner will decide which items have the highest priority, and the Development Team decides

the amount of work they feel they can handle during the sprint. Items pulled from the Product Backlog will be placed in the Sprint Backlog (James 2012).

2.2.4 ***Event***

- *Held once at the beginning of the sprint.*

2.3 Daily Standup

2.3.1 **Who**

- Development Team, Testing Team, CI/CD Team, Product Owner (if needed)

2.3.2 **Agenda**

- Welcome
- Report of activities

2.3.3 **Purpose**

- *As a rule, the Scrum Development Meeting must be short. In fact, Daily Scrum Meetings must be 15 minutes or less.* Each team member will give a brief report of yesterday's activities, what they will do today, and what obstacles they are facing. The Sprint Task List may be updated at this meeting. The Product Owner may attend, but it is preferred that Daily Scrum only consists of the Development Team (James, 2012). *As such, we have chosen to make the Product Owner's attendance optional, only if necessary. In differentiation of our project from SCRUM, each of our three teams will meet in their own standup meetings, rather than one large meeting with all development teams.*

2.3.4 ***Event***

- *Recurring, held every day in a designated, unchanging time and place.*

2.4 Infrastructure and CI/CD Review

2.4.1 ***Who***

- *CI/CD Team, Project Manager, Software Developers, Software Testers*

2.4.2 ***Agenda***

- *Welcome*
- *Review of CI/CD pipeline health*
- *Discuss automation failures and system stability*

2.4.3 ***Purpose***

- *All automated processes will inevitably fail or produce errors. It is one of many duties of the CI/CD Team and the Testing Team to fix these incidents, and the job of the Development Team to assist in creation of error-free software. At this meeting, the CI/CD team will report any errors they have encountered in the automation and what fixes were implemented. Additionally, plans will be devised to prevent errors in the future. Because the CI/CD Pipeline is so crucial to our project, we have decided that a weekly meeting devoted entirely to discussion of the pipeline and any related errors would be beneficial to both our own teams and stakeholders.*

2.4.4 ***Event***

- *Weekly.*

2.5 Sprint Retrospective Meeting

2.5.1 **Who**

- All employees.

2.5.2 **Agenda**

- Welcome
- Reflection of previous sprint
- Planning for future challenges

2.5.3 **Purpose**

- *Every sprint will end with a Sprint Retrospective Meeting.* The purpose of a retrospective meeting is to reflect on the previous iteration and create solutions for flaws and areas of improvement (Tripani, 2023). As such, the previous sprint's work will be discussed, and areas of concern will be noted. Plans will be created to adapt to challenges (James, 2012). *This will be done through collection of team feedback, allowing all team members the opportunity to share opinions on the previous sprint.*

2.5.4 **Event**

- *This meeting will be held at the conclusion of each iteration.*

3. DOCUMENTS

3.1 *Products Requirement Document*

3.1.1 ***Author***

- *Product Owner, ScrumMaster, Stakeholders*

3.1.2 ***Intended Audience***

- *All employees and stakeholders*

3.1.3 ***Purpose***

- *A Product Requirements Document (PRD) will be created before the project is started. In Radigan's words, "It outlines the product's purpose, its features, functionalities, and behavior".* A PRD will determine the specifics of the project, assumptions, and business objectives. It will also include user stories, which will be beneficial in the creation of Sprint Tasks and PBIs (Radigan, n.d). *Though the Products Requirement Document is not required in James's Scrum Reference Card, we have chosen to implement a Products Requirement Document for the mutual benefit of our stakeholders and company.*

3.1.4 ***Deadline***

- *This document will be created at the Product Requirements Meeting.*

3.1 Product Backlog

3.2.1 **Author**

- Anyone, including the stakeholders, may create an item for the Product Backlog. The log is maintained by the Product Owner.

3.2.2 **Intended Audience**

- All employees and stakeholders.

3.2.3 **Purpose**

- *The Product Backlog is a collection of Product Backlog Items. In the backlog, items are listed in order of priority and ROI (James 2012).* The Product Backlog allows for simple, easily readable system design and a more loosely defined documentation strategy.

3.2.4 **Deadline**

- *This document will be created on the first day of the project at the first Sprint Planning Meeting. It will be continuously maintained by the Product Owner and refactored at the Backlog Refinement Meeting.*

3.3 Product Backlog Items

3.3.1 **Author**

- Anyone, including the stakeholders, may create a Product Backlog Item.

3.3.2 **Intended Audience**

- All employees and stakeholders, particularly the Development Team.

3.3.3 **Purpose**

- *Product Backlog Items (PBI) are often written like User Stories. They define a task to be completed and the necessary acceptance criteria. The Development Team will estimate how much time and manpower is required to complete each item (James 2012).* These items are similar to User Stories as detailed in the XP methodology. *In Salimi's words, "Product Backlog Items ensure that the customer is getting the right product to meet their needs" (Salimi, n.d.)*

3.3.4 **Deadline**

- *These documents will first be created on the first day of the project at the first Sprint Planning Meeting. After this, they will be re-evaluated, moved through the backlog, and even created daily.*

3.4 Sprint Backlog

3.4.1 **Author**

- PBIs found in the Sprint Backlog will be negotiated by the Product Owner and Development Team.

3.4.2 **Intended Audience**

- All employees and stakeholders, particularly the Development Team.

3.4.3 **Purpose**

- Not to be confused with the Product Backlog, the Sprint Backlog is a collection of Product Backlog Items to be completed during the current Sprint. It details the status and requirements of tasks to be completed and finished tasks (James 2012).

It is visible to the team and discussed during Daily Standup Meetings. This artifact is similar to the Product Backlog, but refers more specifically to *only* PBIs to be worked on during the current sprint.

3.4.4 **Deadline**

- *This document will be created at the first Sprint Planning Meeting. It will be re-evaluated at each successive Sprint Planning Meeting.*

3.5 Sprint Task

3.5.1 **Author**

- Development Team

3.5.2 **Intended Audience**

- All employees and stakeholders, particularly the Development Team.

3.5.3 **Purpose**

- If PBIs are the what of a problem, Sprint Tasks are the how. Each Sprint Task is a broken down PBI requiring one day or less of work. Remaining effort will be re-estimated daily. Teams are expected to collaborate towards completion of each Task (James 2012).

3.5.4 **Deadline**

- *These documents will first be created on the first day of the project at the first Sprint Planning Meeting. After this, they will be re-evaluated, moved through the backlog, and even created daily.*

3.8 Test Plan

3.8.1 ***Author***

- *Testing Team, Technical Writer*

3.8.2 ***Intended Audience***

- *All employees and stakeholders*

3.8.3 ***Purpose***

- *For the sake of organization and thorough documentation, all tests created for this project will be organized into a test plan document. This document will be updated daily as new tests are run, completed, and created. This document will serve as a guide for all tests created in each iteration. Though a Test Plan may not explicitly be required in some interpretations of DevOps, we think it is wise to keep records of all tests performed on the software.*

3.8.4 ***Deadline***

- *This document will be created at the beginning of each iteration. It will also be updated daily as tests are run and new tests are created.*

3.8 CI/CD Pipeline Documentation

3.8.1 **Author**

- CI/CD Team, Project Manager, Technical Writer

3.8.2 **Intended Audience**

- All employees and stakeholders

3.8.3 **Purpose**

- **It is imperative that automated software integration flows are documented (Stahl & Bosch, 2014).** Therefore, our team will accurately document all build, test, staging, and deployment automation. This documentation will serve as a guide to the numerous automated processes involved in the development of the product. *This documentation will be created at the beginning of the project during initial setup of the CI/CD pipeline and updated frequently with new information as it arises, especially at the CI/CD Meetings..*

3.8.4 **Deadline**

- *This document will be created at the beginning of the project. It will be updated continuously as new automated processes are used.*

3.8 Infrastructure & Deployment Strategy

3.8.1 **Author**

- CI/CD Team, Project Manager, Technical Writer

3.8.2 **Intended Audience**

- All employees and stakeholders

3.8.3 **Purpose**

- **Staging, development, testing, and production environments are often different. Lack of documentation in these areas frequently causes development issues. A plan for deployment must be created in order for smooth continuous releases of software (Zeleke & McCollum, 2021).** Therefore, our team will accurately document all infrastructure and setup necessary. Additionally, this documentation will also define software security measures and the monitoring tools utilized in our DevOps process. This documentation will serve as a guide to the infrastructure created and the deployment and operation processes. *This documentation will be created at the beginning of the project and updated frequently with new information as it arises.*

3.8.4 **Deadline**

This document will be created at the beginning of the project. It will be updated continuously as new automated processes are used.

4. ROLES

4.1 Project Manager (1 person)

4.1.1 **Who**

- Oliver

4.1.2 **Qualifications**

- Significant experience in development, testing, and especially management. The Project Manager must be capable of communicating with clients and administrating the software development process.

4.1.3 **Responsibilities**

- The project manager should manage both the development of the product and client relations. *They should also assist in drafting documentation and designing the finished product. The Project Manager will preside over the entire development process and aim the project towards completion (Ovcharenko 2024).*

4.2 UX Designers (2 people)

4.2.1 **Who**

- Ursula and Xavier.

4.2.2 **Qualifications**

- College degree in or related to graphic design or visual media. Experience in the design industry.

4.2.3 **Responsibilities**

- The UX Designers will handle all design work for the entire project. They will create wireframes and mockups using design principles. They will also ensure the end-user portions of the product are visually pleasing. *In other words, they are responsible for the visual design of the software (Ovcharenko, 2024).*

4.3 Technical Writer (1 person)

4.3.1 **Who**

- Teri.

4.3.2 **Qualifications**

- College degree in or related to Technical Writing, Communications, or English.

4.3.3 **Responsibilities**

- The Technical Writer will create all of the text in the product. They are responsible for, alongside the coding team, creating and proofreading all text that will be displayed to the user. The Technical Writer will also proofread code comments for accuracy. Most importantly, the Technical Writer will draft, generate, edit, and proofread all documentation and required documents. They are responsible for high-quality, readable documentation.

4.4 Software Developers (4 people)

4.4.1 **Who**

- Claire, Emily, Larry, Grace

4.4.2 **Qualifications**

- College degree or working experience in software engineering, computer science, design, or a related field.

4.4.3 **Responsibilities**

- The Coding Team Member is responsible for developing the code for the project. They will collaborate with one another to create a working product. **All developers MUST be well-practiced and capable with version control systems. A staple of DevOps development is frequent version control (Kim et al. 2016).** *To fulfill this requirement, our project will use Git to keep all code, artifacts, and automation in a repository.*

4.5. Software Testers (4 people)

4.5.1 **Who**

- Frank, Holly, Keith, Doug

4.5.2 **Qualifications**

- College degree or working experience in software engineering, computer science, design, or a related field.

4.5.3 **Responsibilities**

- **All software should be thoroughly tested and checked for quality before it is released to production. It was mentioned by De Bayser that automated processes help in the testing of deployment and execution of software (De Bayser et al, 2015).** *Our project will use the automated testing software Selenium for automated testing procedures.*

4.5. CI/CD Team (4 people)

4.5.1 **Who**

- Abe, Britney, Jack, Ingrid

4.5.2 **Qualifications**

- College degree or working experience in software engineering, computer science, design, or a related field.

4.5.3 **Responsibilities**

- **It is crucial to the DevOps methodology that the CI/CD (Continuous Integration / Continuous Deployment) team is involved in product development, testing, and deployment. Members of the CI/CD team must be able to quickly detect and recover when incidents in the service appear, and they are responsible for deployment and monitoring of the software (Kim et al. 2016). Most importantly, they are responsible for the design and maintenance of the CI/CD pipeline (Shahin et al. 2017).** This helps to create a more robust product that remains serviceable through the product's lifespan and a

positive feedback loop. *Our CI/CD Team will use the following tools to create the deployment pipeline* **(Sourced from Shahin et al. 2017):**

I. *Version Control System - Git*
II. *Code Management and Analysis - SonarQube*
III. *Build System - Make*
IV. *CI Server - Jenkins*
V. *Testing - TestLink*
VI. *Configuration and Provisioning - Yum*
VII. *CD Server - Jenkins*

5. CHECKPOINTS

5.1 *Product Requirements Completed*

5.1.1 Effort Estimation

- *1 week after the project starts.*

5.1.2 Exit Criteria

- *Project and software requirements have been identified and documented through collaboration with stakeholders. A Project Requirements Document has been created and to-be implemented user stories for this project have been defined. This checkpoint will only be completed upon creation of the Project Requirements Document. Though the Products Requirement Meeting is not required in some interpretations of DevOps, we have chosen to implement a Products Requirement Document and Meeting for the mutual benefit of our stakeholders and company.*

5.2 Sprint Planning Completed

5.2.1 Effort Estimation

- *Recurring, will be completed on the first day of each sprint.*

5.2.2 Exit Criteria

- **A plan for the subsequent sprint has been completed including the Sprint Backlog. PBIs from the Product Backlog have been converted into Sprint Tasks, and the Development Team has negotiated the amount of work they**

feel they can handle during the sprint. This checkpoint can only be completed after the Sprint Planning Meeting.

5.3 Sprint Development Completed

5.3.1 Effort Estimation

- *Recurring - but at most, this will take 2 weeks per iteration.*

5.3.2 Exit Criteria

- **All Sprint Tasks for this iteration have been completed. In other words, this iteration's development work is finished.**

5.4 Automated Testing Completed

5.4.1 Effort Estimation

- *This checkpoint will be progressing during the development checkpoint. As such, it will end at the same time as the development checkpoint - 2 weeks at maximum.*

5.4.2 Exit Criteria

- **Automated Testing is crucial to the DevOps method of developing software (Stahl & Bosch 2014). In this checkpoint,** *all defined test cases, as defined in the test plan document, are successful and free from errors.* **The code is clean, readable, and deployable. It has been thoroughly debugged and rigorously tested. Additionally, the code must fulfill all safety and security requirements.**

5.5 Deployment to Staging

5.5.1 **Effort Estimation**

- *At maximum, 2 weeks after iteration starts.*

5.5.2 **Exit Criteria**

- **Software should be staged before it is deployed (Shahin et al. 2017). Inside of the staging environment, all core features will be tested to ensure they run properly.** *All tests will be documented in the Test Plan document.* **This process will ensure there are no major defects with the software before it is pushed to deployment.**

5.5 Production Deployment

5.5.1 **Effort Estimation**

- *At maximum, 3 weeks after iteration starts.*

5.5.2 **Exit Criteria**

- **All security checks and tests have passed, and client approval has been received from client representatives. With all development and testing complete, the software for this iteration is released. This checkpoint will only be fulfilled upon successful release of this iteration's software.**

5.6 Retrospective Completed

5.6.1 **Effort Estimation**

- *This checkpoint will be reached at the conclusion of the iteration - at maximum, 3 weeks from iteration start.*

5.6.2 **Exit Criteria**

- **The retrospective meeting has been completed, and all team members have reviewed the iteration's successes and failures. This checkpoint will only be fulfilled upon completion of the Sprint Retrospective Meeting.**

5.5 Monitoring and Continuous Improvement of Deployed Software

5.5.1 **Effort Estimation**

- *Ongoing, to begin after first production deployment.*

5.5.2 **Exit Criteria**

- **This is perhaps the most integral part of the DevOps process. Released software should be monitored and continuously improved to ensure stability throughout the product's lifecycle, through "continuous feedback, quick response to changes and using automated delivery pipelines resulting in reduced cycle time" (Jabbari et al. 2016). While in production, the software will be continuously monitored for incidents. Automated alerts will inform of both security and performance issues. Additionally, the software will be subject to regular maintenance, fixes, and upgrades. Unlike other checkpoints, monitoring and continuous improvement of deployed software is an ongoing process that will continue throughout the entire lifetime of the product.**

5.7. Timeline

Event	Date
Product Requirements Completed *Deliverable: Product Requirements Document*	1 week after the project starts.
Sprint Planning Complete	1 day after iteration starts.
Sprint Development Completed Deliverable: Prototype of the iteration's software	At maximum, 2 weeks after iteration starts.
Automated Testing Completed	At maximum, 2 weeks after iteration starts.
Deployment to Staging Complete	At maximum, 2 weeks after iteration starts.
Production Deployment Deliverable: One iteration's completed, working software	At maximum, 3 weeks after iteration starts.
Iteration Retrospective Complete	At maximum, 3 weeks after iteration starts.
Monitoring and Continuous Improvement of Deployed Software	Ongoing.

6. ANALYSIS

In the following section, we will list the strengths and weaknesses of the DevOps development methodology, following the analysis made by Veritis Group (2025). Then we will address a final summary and reflect on our implementation of the methodology.

6.1 Strengths/Advantages

“Everything has its pros and cons. While DevOps has set out to take the agile process to new heights, it has disadvantages. However, many have acknowledged that DevOps outweighs its cons with its pros” (Veritis Group, 2025).

1. **Faster Time-to-Market** - Using the DevOps methodology, utterly functional software can be developed faster. The advocated automation saves a ton of time. Operations that once required numerous handoffs and took weeks to complete now only take a few keystrokes or the execution of a script.

2. **Stunted Expenses** - With automation finishing crucial tasks, there are a few aspects that one doesn’t have to be bothered about. This is primarily due to how advanced automation has become in recent times. Whether bug detection, update deployment, or the execution of mundane tasks, automation has enough in its arsenal to reduce expenses. While automation may have its fair share of teething troubles, it brings along many benefits in the long run. On average, it was generally observed that companies could reduce their overheads by 20% with DevOps integration.

3. **Seamless Workflow** - Let’s face it: waterfall siloed the team members quite rigidly. DevOps brings together the team in a much more effective way. Coordination is the key

to productivity, and DevOps principles ensure coordination and collaboration pave the way for a seamless workflow.

4. **Bugs Quashed** - Automatic testing, one of the principles of DevOps, allows the project members to fix the bugs fast. As a result, DevOps meets quality standards faster.

5. **Better Integration** - Automation helps you deploy updates faster than before. As DevOps brings along automation, project members have the time to develop DevOps solutions that can support various devices. The automation advantage allows you to schedule the updates for automatic rollouts, which eliminates the problematic aspects of deploying the DevOps solutions.

6. **Planning Improvised** - As DevOps breaks down the project delivery as iterative updates, the project leads can create a practical roadmap of the deliverables, which includes important aspects such as development, testing, and deployment. The roadmap decides the solution's life cycle, which every key player, including the stakeholders, should include at this stage.

6.2 Weaknesses/Disadvantages

DevOps disadvantages, in opinion of Veritis Group (2025), are more challenges that need to be overcome than real impediments.

1. **Complexity** - Implementing DevOps platforms can frequently increase the complexity of a firm's IT environment. By integrating many tools and technologies into the software development process, companies can create a more complicated, challenging production

environment to manage and troubleshoot. DevOps can also force businesses to spend more money on hardware and software resources, which raises complexity and expenses.

2. **No Standardized Version** - DevOps is currently not standardized across the entire industry. As a result, DevOps companies must develop unique workflows and toolkits, which can be time-consuming and expensive. Additionally, a deficiency of uniformity may cause uncertainty among staff members regarding the best ways to apply DevOps principles.

3. **Teething problems** - The increased adoption of DevOps has resulted in a lack of skilled DevOps engineers and specialists. As a result, corporations are frequently forced to hire inexperienced workers or consultants as they scramble to integrate DevOps solutions in the initial phase. Unfortunately, this hotchpotch execution leads to software issues and quality degradation.

4. **Early Adoption Overheads** - Adoption of DevOps can frequently result in higher expenditures for enterprises. Enterprises can considerably increase their IT costs by investing in extra hardware and software resources and hiring skilled DevOps specialists. Furthermore, the sophistication of a DevOps system frequently causes issues with application performance and dependability.

5. **Technical Dissonance** - DevOps solutions also bring with them a slew of technological difficulties. One of the most pressing is the need to standardize and automate procedures and technologies throughout the business. This may be challenging, especially in firms with several apps and DevOps platforms. Another difficulty is incorporating DevOps solutions into a current system without sacrificing safety or speed.

6. **Operational Impediments** - Finally, the DevOps approach might be challenging to implement from an operational standpoint. One of the most difficult tasks is managing deployments and rollbacks regulated and safely, especially in a big, complicated manufacturing setting. Monitoring and logging are also crucial for DevOps success, but they can be challenging to set up and manage.

6.3 Summary & Reflection

Following the previous analysis, we recognize that the DevOps methodology could align well with our team's needs, though some adjustments may be necessary based on the size and scope of our project. With this in mind, our objective is to reintroduce the Scrum framework outlined in *REPORT 08 - Scrum Plan* (Nelson & Mercado, 2025) while integrating DevOps practices.

This approach would involve incorporating pipeline workings and automation processes to enhance the application's development lifecycle, along with team collaboration practices central to DevOps.

Additionally, we know that security considerations remain a key concern. To address this, we may explore DevSecOps at a later stage—once the team has gained familiarity with DevOps principles. This would require further analysis, potentially leading to a separate report examining the various aspects of DevSecOps, as outlined in the appendix of this document.

Regardless, we believe DevOps is a possible methodology which could be used to some success in our project, but it is not perfect. We believe that methodologies such as XP or Scrum would be better suited for the creation of our product, simply because the size and power of the DevOps methodology may be too great for a simple app such as the one requested by B.A.N.K. DevOps could work, but it is not ideal for our scenario.

7. REFERENCES

- Beck, et al. (2001) Manifesto for Agile Software Development. Available at: <https://agilemanifesto.org/>. (Accessed: 26 February 2025).
- Codecademy (2025). *Introduction to devops: Resiliency cheatsheet* (no date) *Codecademy*. Available at: <https://www.codecademy.com/learn/introduction-to-dev-ops/modules/resiliency/cheatsheet> (Accessed: 15 March 2025).
- De Bayser et al. (2015). *ResearchOps: The case for DevOps in scientific applications, 2015 IFIP/IEEE International Symposium on Integrated Network Management.* Available at: <https://www.researchgate.net/publication/283757158_ResearchOps_The_case_for_DevOps_in_scientific_applications>. (Accessed: 12 March 2025).
- GitLab (2023) *What is devops??*, *GitLab*. Available at: <https://about.gitlab.com/topics/devops/> (Accessed: 15 March 2025).
- J. G. (2013) *Agile methodology*, *Agile Methodology*. Available at: <https://oracleebspro.blogspot.com/2013/02/agile-methodology.html> (Accessed: 26 February 2025).
- Jabbari, R. *et al.* (2016) *What is DevOps?: A Systematic Mapping Study on Definitions and Practices*, *ResearchGate.net*. Available at: <https://www.researchgate.net/publication/308857081_What_is_DevOps_A_Systematic_Mapping_Study_on_Definitions_and_Practices> (Accessed: 15 March 2025).
- Jacobs, M. *et al.* (2023) *What is devops? - Azure DevOps*, *Azure DevOps | Microsoft Learn*. Available at: <https://learn.microsoft.com/en-us/devops/what-is-devops> (Accessed: 13 March 2025). It was not possible to find out the real name of one of the authors, whose GitHub username is [KathrynEE].

- James, M. (2012) *Scrum reference card*. Available at: <https://scrumreferencecard.com/scrum-reference-card/> (Accessed: 26 February 2025).
- Kim, G. et al. (2016). *The DevOps Handbook*. Portland, OR: IT Revolution Press.
- Kolodiy, R. (2023) *DevOps VS DevSecOps: Integrating security into the CI/CD pipelines*, *TechMagic*. Available at: <https://www.techmagic.co/blog/devops-vs-devsecops> (Accessed: 14 March 2025).
- Krammer, N. (2024) *CI/CD Pipeline Feedback Loops: Best practices.* Available at: <https://daily.dev/blog/cicd-pipeline-feedback-loops-best-practices> (Accessed: 14 March 2025).
- Krammer, N. (2024a) *CI/CD Pipeline Orchestration: Complete Guide 2024*. Available at: <https://daily.dev/blog/cicd-pipeline-orchestration-complete-guide-2024> (Accessed: 15 March 2025).
- Kukushkina, N. (2023) *State of DevOps vs DevSecOps – What's the Difference, DigitalMara*. Available at: <https://digitalmara.com/blog/state-of-devops-vs-devsecops-whats-the-difference/> (Accessed: 13 March 2025).
- Microsoft Azure DevOps Tutorial (2025) *DevOps Tutorial | Microsoft Azure.* Available at: <https://azure.microsoft.com/en-us/solutions/devops/tutorial#> (Accessed: 13 March 2025).
- Nelson L. & Mercado C. (2025) *REPORT 08 - Scrum Plan*. Available at: <https://carloswm85.github.io/documents/learning/dev-methodologies/2025-Report_08_SCRUM_Plan.pdf> (Accessed: 13 March 2025)
- Ovcharenko, D. (2024). *Perfect Software Development Team: Roles and Responsibilities, Alcor.* Available at: <https://alcor-bpo.com/10-key-roles-in-a-software-development-team-who-is-responsible-for-what/> (Accessed: 06 March 2025).

- Queen, C. (2024) *DevOps vs. DevSecOps: Understanding the Difference*, *DevOps vs DevSecOps: What's the difference? | CrowdStrike*. Available at: <https://www.crowdstrike.com/en-us/cybersecurity-101/cloud-security/devops-vs-devsecops/> (Accessed: 15 March 2025).

- Radigan, D. (n.d). *How to create a product requirements document (PRD), Atlassian.* Available at: <https://www.atlassian.com/agile/product-management/requirements>. (Accessed: 07 March 2025).

- Salimi, S (n.d). *Product backlog item (PBI), Agile Academy*. Available at: <https://www.agile-academy.com/en/agile-dictionary/product-backlog-item-pbi/#:~:text=A%20Product%20Backlog%20Item%20.> (Accessed: 07 March 2025).

- Shahin, M. et al (2017). *Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices, IEEE Access.* Available at: <https://www.researchgate.net/publication/315381994_Continuous_Integration_Delivery_and_Deployment_A_Systematic_Review_on_Approaches_Tools_Challenges_and_Practices> (Accessed: 12 March 2015).

- Ståhl, D. & Bosch, J. (2014). *Automated software integration flows in industry: A multiple case study, 36th International Conference on Software Engineering, ICSE Companion 2014 - Proceedings*. Available at: <https://www.researchgate.net/publication/266656876_Automated_software_integration_flows_in_industry_A_multiple-case_study>. (Accessed: 12 March 2025).

- SUDO Consultants (2024) *Ultimate Guide to Monitoring & Logging in DevOps*. Available at: <https://sudoconsultants.com/ultimate-guide-to-monitoring-and-logging-in-devops-concepts-types-best-practices/> (Accessed: 15 March 2025).

- Trapani, K. (2023) *How to hold effective retrospectives without wasting time, Cprime*. Available at: <https://www.cprime.com/resources/blog/how-to-hold-effective-retrospectives-without-w

asting-time/#:~:text=Retrospective%20meetings%20are%20a%20key,Sprint%20better%20than%20the%20last>. (Accessed: 14 February 2025).

- Veritis Group (2025) *Explained: Pros & Cons of DevOps Methodology and its Principles*. Available at: <https://www.veritis.com/blog/explained-pros-and-cons-of-devops-methodology-and-its-principles/> (Accessed: 15 March 2025).

- Wasike, B. (2023) *The Ultimate Guide to DevOps: Best Practices, Tools, and Application in Software Development*, *DEV Community*. Available at: <https://dev.to/bravinsimiyu/the-ultimate-guide-to-devops-best-practices-tools-and-application-in-software-development-2bj1> (Accessed: 15 March 2025).

- Zeleke, A. & McCollum, W. (2021). *Determinants of effective change management for software deployment projects, International Journal of Applied Management & Technology, Walden University.* Available at: <https://scholarworks.waldenu.edu/cgi/viewcontent.cgi?article=1458&context=ijamt>. (Accessed: 12 March 2025).

8. APPENDIX

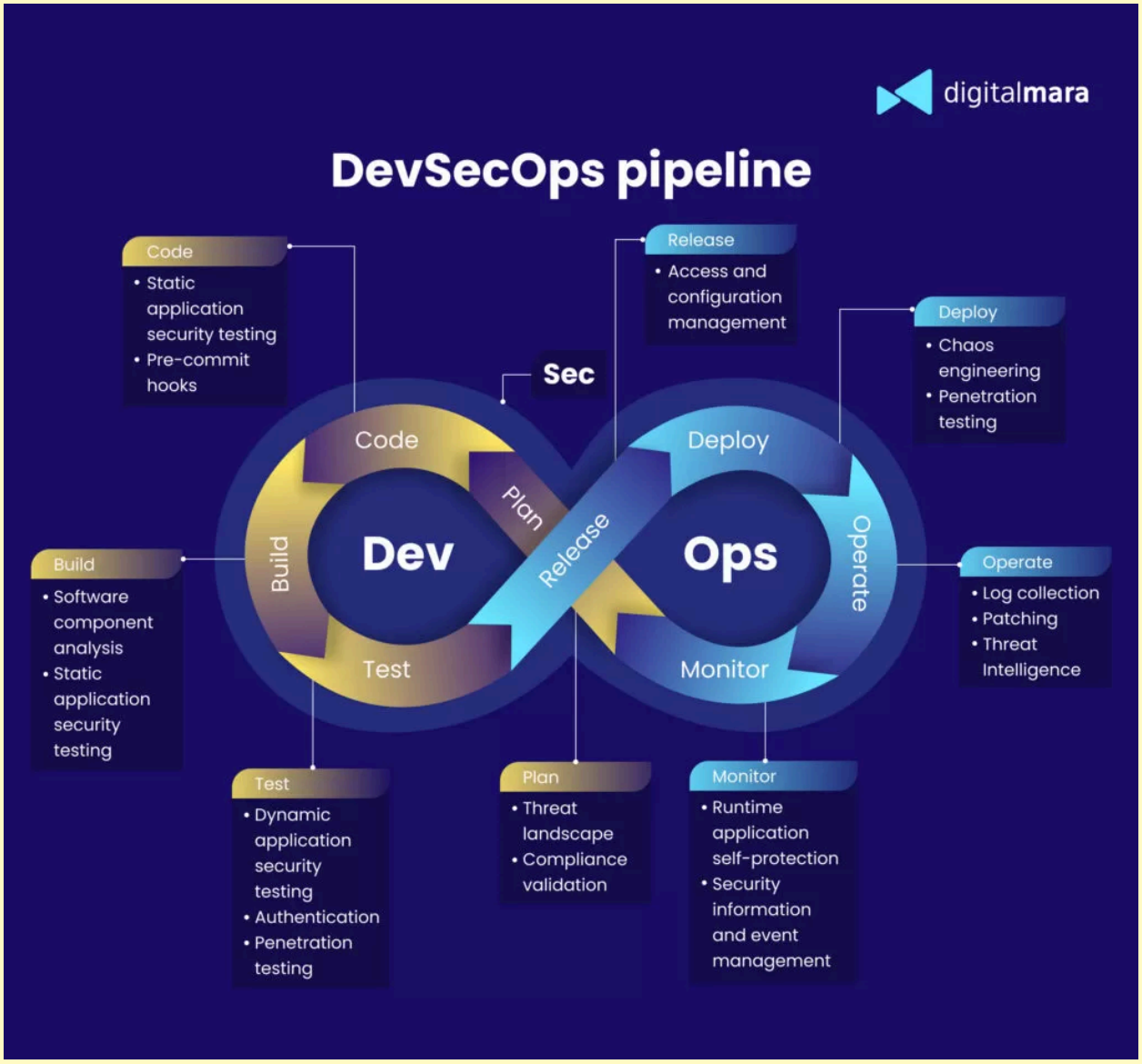

Figure 2 *- DevSecOps Pipeline description (Kukushkina, 2023).*

Criteria		DevOps	DevSecOps
Purpose	Focus on improving quality and increasing the speed of software development and delivery.	Yes	Yes + applying security at every stage of the development lifecycle.
Team	Development and operations teams are working together.	Yes	Yes + security team
Processes	Continuous integration, continuous delivery and continuous deployment workflows.	Yes	Yes + additional security-related processes.
Tools	Git, Terraform, Kubernetes, Ansible, Docker, Jenkins, Nagios	Yes	Yes + security-specific tools like Veracode, Burp Suite, OWASP ZAP Proxy
Security	When security issues are resolved?	At the end of the development process.	Throughout the entire development process from start to finish.
Vulnerabilities	When vulnerabilities are eliminated throughout the development lifecycle?	Not always	Always

Figure 3 *- DevSecOps is not just a supplement to the traditional DevOps approach. It's a special set of practices that differ, though they overlap. DevOps focuses mainly on efficiency and technical aspects, while the key concern of DevSecOps is security. (Kukushkina, 2023).*